

MultiSet Integration Report

1. MultiSet Response Delay

1.1 Observed Delay Breakdown

MultiSet <-(~1000ms)- JJ Glasses -(~300ms)-> Firebase
AR Glasses Apply Localization Time: 0.1ms ~ 0.2ms

1.2 Interpretation

- MultiSet localization round-trip time is approximately ~1000 ms.
- Firebase data fetch time is approximately ~300 ms.
- AR glasses localization pose apply time is approximately 0.1 ms ~ 0.2 ms.

From the current measurements, the dominant delay comes from network-side localization and cloud data retrieval, not from pose application inside Unity.

In practice, the end-to-end wait perceived by the user is roughly ~1.3 s before localized map context is fully imported from Database, which means the first delay time is max to 1.3s delay, but the average delay is about 0.8~1.0s for waiting the MultiSet API in full experience.

While the actual transform application to the AR scene is nearly instantaneous.

2. MultiSet Localization Logic

2.1 Single Localization Response Content

In the current single-frame localization flow, one image is sent to MultiSet and the response mainly includes:

- poseFound: whether localization succeeded.
- position: localized camera position in MultiSet space.
- rotation: localized camera rotation in quaternion form.
- confidence: confidence score of the localization result.
- mapIds: matched map identifier list.

After localization is applied, the system also keeps runtime data such as:

- success
- mapId

- `mapName`
- `confidence`
- `localizationTimeSeconds`
- `localizedCameraPosition`
- `localizedCameraRotation`
- `mapSpacePosition`
- `mapSpaceRotation`
- `errorMessage` on failure

2.2 Current Pose Application Strategy

The system does not move the `Player` directly. Instead, it calculates the localization transform and applies it to `mapSpace`, so the AR camera tracking can remain stable while the scene content is aligned to the real world.

2.3 Why Transitioning Scene Content Is Better Than Moving Player

Compared with changing the `Player` transform, applying localization to `mapSpace` has these advantages:

- It keeps AR tracking more stable.
- It reduces conflicts with `ARCore` / `ARFoundation` tracking ownership.
- It avoids obvious viewpoint jumps on AR glasses.
- It makes relocalization easier because only the scene root needs to be re-aligned.
- It is easier to manage localized meshes, POIs, and map content under one root object.

2.4 Built-in Optimization in the Current System

The current `SingleFrameLocalizationManager` already provides:

- Multiple localization attempts with best-result selection by highest confidence.
- Optional confidence threshold filtering through `confidenceCheck` and `_confidenceThreshold`.
- Background localization support.
- Relocalization support when AR tracking changes or is lost.
- Blur detection and retry logic before sending poor-quality frames.
- Silent retry mode for the first localization until success.
- Support for AR-session-based input and externally supplied image/intrinsics input.
- Running average localization time tracking for debug UI.

3. MultiSet SDK Decompilation

3.1 Why Decompile Was Needed

Originally, MultiSet provided the integration mainly as a DLL-based plugin. Under that structure, it was not practical to directly modify the plugin script behavior, internal localization flow, or ARCore-related dependency handling from within this Unity project.

To make the SDK maintainable inside JJ_VPS, the DLL was decompiled with dnSpy into:

[Assets/MultiSet/Script/](#)

The decompiled output was then repaired and adapted with Codex collaboration so that the scripts could compile and run inside the project again.

3.2 Current Decompile Scope

Based on the current project state, the decompiled MultiSet script directory contains 99 C# scripts under `Assets/MultiSet/Script/`.

This gives the project direct control over:

- localization request / response handling
- map and mapset loading logic
- mesh download flow
- on-device localization bridge code
- editor and runtime integration behavior

3.3 Scripts Modified So Far

From the current observable working tree, the scripts with active modifications are:

- `Assets/MultiSet/Script/SingleFrameLocalizationManager.cs`
- `Assets/MultiSet/Script/MapMeshHandler.cs`

These modifications are meaningful because `SingleFrameLocalizationManager.cs` is the core runtime path for single-frame localization and pose application, while `MapMeshHandler.cs` is directly related to post-localization map mesh retrieval and scene attachment.

3.4 Practical Outcome

After decompilation, the MultiSet SDK is no longer a black-box dependency inside this project. The team can now:

- inspect and adjust localization behavior directly
- decouple or soften original ARCore assumptions when needed

- fix compile/runtime issues inside the Unity project
- extend the SDK behavior to better fit the JJ glasses integration flow

This significantly improves maintainability, debugging capability, and future integration flexibility for JJ_VPS.